2025년 홍익대학교 AI 기술 트렌드 분석 및 신규 AI 서비스 프로젝트 제안서

1. 개요

본 문서는 2025년 홍익대학교 컴퓨터공학과 졸업 프로젝트 기술 백서와 최신 생성형 AI 트렌드를 분석하여, 학술적 깊이와 실용성을 겸비한 신규 졸업 프로젝트 아이디어 **10**개를 제안합니다.

1.1 제안 조건

- 기술 스택: Express + Node.js + MongoDB (+ Python/LLM API 연동)
- 지도교수 적합성: 운영체제 전공 교수님 → 시스템 관점에서의 AI 서빙, 리소스 관리, 스케줄링 개념 융합
- 난이도: 1인 개발 가능하면서도 학술적 의미가 있는 수준
- 차별화: 단순 API 호출이 아닌, OS/시스템 이론을 AI 도메인에 적용

1.2 2025년 생성형 AI 핵심 트렌드

> **"2025년은 생성형 AI가 실험 단계를 벗어나 실전 비즈니스 가치를 창출하는 해"**

트렌드	설명	학술적 연구 포인트
🤖 **Agentic AI**	스스로 계획·실행·최적화하는 자율형 AI	에이전트 아키텍처, 멀티 에이전트 협업
📚 **RAG**	외부 지식을 검색하여 LLM 정확도 향상	검색 알고리즘 비교, 하이브리드 검색
⚡ **LLM 서빙 최적화**	추론 비용/지연 시간 최적화	스케줄링, 캐싱, 배칭 알고리즘
🔒 **온디바이스 AI**	경량 모델 + 개인정보 보호	엣지 컴퓨팅, 리소스 제약 환경
✅ **AI 신뢰성**	환각 탐지, 팩트체크, 출처 추적	NLI, 자동 검증 파이프라인

---

2. 프로젝트 아이디어 제안 (10개)

◆ 전체 비교표

#	프로젝트명	OS 연관성	AI 깊이	실용성	난이도
1	LLM 추론 요청 스케줄러	⭐⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	중상
2	하이브리드 RAG 검색 엔진	⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	중상
3	한국어 규정/법령 RAG 시스템	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	중
4	LLM 응답 시맨틱 캐싱	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐	중
5	코드 취약점 분석 + LLM 설명	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐	중상
6	분산 LLM 추론 게이트웨이	⭐⭐⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐	중
7	LLM 환각 탐지 시스템	⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	상
8	시스템 장애 로그 분석 AI	⭐⭐⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐	중
9	로컬 프라이버시 AI 에이전트	⭐⭐⭐	⭐⭐⭐⭐	⭐⭐⭐	중상
10	API 문서 자동 생성 시스템	⭐⭐	⭐⭐⭐	⭐⭐⭐⭐⭐	중

1️⃣ LLM 추론 요청 스케줄러 ⭐ 운영체제 교수 처적 추천

문제 정의
- 다수 사용자가 LLM API를 동시 호출 시 비용 폭증 + 응답 지연
- 모든 요청을 동일하게 처리하면 중요 요청도 대기

해결 방안
OS 프로세스 스케줄링 알고리즘(MLFQ, Priority Queue, Weighted Fair Queue)을 LLM 요청에 적용 - 우선순위, 토큰 예산, 대기시간 기반 동적 스케줄링 - 요청 배칭(batching)으로 처리량 최적화

핵심 기술
```
Express Proxy Server, 요청 큐잉 (Bull/BullMQ)
우선순위 알고리즘 구현, MongoDB (요청 이력/통계)
LLM API 연동, 실시간 대시보드 (Socket.io)
```

학술적 가치
- **OS** 스케줄링 이론을 신규 도메인(LLM 서빙)에 적용
- 알고리즘별 성능 비교 실험 (처리량, 평균 대기시간, 공정성)
- 논문 작성 가능: "LLM 추론 요청을 위한 적응형 스케줄링 알고리즘"

예상 개발 기간
- MVP: 5주 / 완성본: 10주

2️⃣ 하이브리드 RAG 검색 엔진 (Dense + Sparse + Re-ranking)

문제 정의
- 벡터 검색(Dense)만으로는 키워드 매칭 부족
- BM25(Sparse)만으로는 의미론적 유사성 파악 부족

해결 방안
Dense(벡터)와 Sparse(BM25) 검색을 결합한 하이브리드 검색 - RRF(Reciprocal Rank Fusion)로 결과 통합 - Cross-encoder Re-ranking으로 최종 순위 결정

핵심 기술
```
MongoDB Atlas Vector Search, Elasticsearch/Meilisearch (BM25)
OpenAI Embeddings, Cross-encoder (sentence-transformers)
RRF 알고리즘, Express API
```

학술적 가치
- 검색 기법별 성능 비교 실험 (Precision, Recall, MRR)
- 도메인별 최적 가중치 연구
- 논문 작성 가능: "한국어 문서에 대한 하이브리드 RAG 검색 성능 분석"

예상 개발 기간
- MVP: 5주 / 완성본: 10주

3️⃣ 한국어 규정/법령 RAG 시스템

문제 정의
- 대학 학칙, 정부 규정, 법령 등 복잡한 문서를 일반인이 이해하기 어려움
- 조항 간 참조 관계(예: "제3조의 규정에 따라...")가 있어 단순 검색으로 불충분

해결 방안
법령/규정 특화 RAG 시스템 구축 - 조항 단위 청킹 + 참조 관계 그래프 구축 - 관련 조항을 함께 검색하여 LLM에 컨텍스트 제공

핵심 기술
```
문서 파싱 (HWP/PDF → 구조화된 JSON)
Knowledge Graph (조항 간 참조 관계)
MongoDB Vector Search, LLM API
Express, React/Vanilla JS
```

학술적 가치
- 도메인 특화 RAG 파이프라인 설계 방법론
- 법률 언어 처리(Legal NLP) 분야 기여
- 논문 작성 가능: "한국어 법령 문서를 위한 그래프 기반 RAG 시스템"

예상 개발 기간
- MVP: 6주 / 완성본: 12주

4️⃣ LLM 응답 시맨틱 캐싱 시스템

문제 정의
- 유사한 질문에 대해 매번 LLM 호출 → 비용 낭비, 응답 지연
- 정확히 동일한 질문만 캐싱하면 효과 낮음

해결 방안
질문을 임베딩하여 의미론적으로 유사한 질문 탐지 → 캐시된 응답 재활용 - 유사도 임계값 조정으로 캐시 히트율/정확도 트레이드오프 관리 - OS의 페이지 교체 알고리즘(LRU, LFU)을 의미론적 캐싱에 적용

핵심 기술

```
OpenAI Embeddings, 코사인 유사도
Redis/MongoDB 캐시, TTL 관리
캐시 교체 알고리즘 (Semantic-LRU 변형)
Express Proxy, 캐시 히트율 모니터링
```

학술적 가치

- **OS** 캐시 이론을 의미론적 도메인에 확장
- 유사도 임계값별 성능 비교 실험
- 논문 작성 가능: "LLM 응답을 위한 시맨틱 캐싱 전략 연구"

예상 개발 기간

- MVP: 4주 / 완성본: 8주

5️⃣ 코드 취약점 정적 분석 + **LLM** 설명

문제 정의

- 정적 분석 도구(ESLint, Semgrep)는 취약점을 발견하지만 왜 위험한지 설명 부족
- 주니어 개발자가 결과를 이해하고 수정하기 어려움

해결 방안

정적 분석 결과 + 코드 컨텍스트를 LLM에 전달 - 취약점 원인, 공격 시나리오, 수정 방법을 자연어로 설명 - OWASP Top 10 등 보안 지식 기반 RAG 연동

핵심 기술

```
AST 파싱 (Babel, Tree-sitter)
Semgrep/ESLint 연동, GitHub API
LLM API + 보안 지식 RAG
Express, MongoDB (분석 이력)
```

학술적 가치

- 보안 분석 자동화 파이프라인 설계
- 설명 가능한 AI(XAI) 원칙 적용
- 논문 작성 가능: "LLM 기반 보안 취약점 설명 시스템"

예상 개발 기간

- MVP: 5주 / 완성본: 9주

6️⃣ 분산 **LLM** 추론 게이트웨이 **(Multi-provider Load Balancing)**

문제 정의

- 단일 LLM 제공자 의존 → Rate Limit, 장애 시 서비스 중단
- 각 제공자별 비용/속도/품질이 다름

해결 방안

OpenAI, Claude, Gemini 등 여러 제공자를 풀로 관리 - 비용/속도/품질 기반 동적 라우팅 - 헬스 체크 + Fallback 로직

핵심 기술

```
API Gateway 패턴, Express Proxy
로드밸런싱 알고리즘 (Weighted Round Robin, Least Connections)
```

```
헬스체크, Circuit Breaker 패턴
MongoDB (요청 로그, 성능 메트릭)
```

학술적 가치
- **OS** 프로세스 스케줄링/로드밸런싱을 AI 서빙에 적용
- 다중 제공자 환경에서의 최적화 전략 연구
- 논문 작성 가능: "이종 LLM 환경을 위한 적응형 로드밸런싱"

예상 개발 기간
- MVP: 4주 / 완성본: 8주

7️⃣ LLM 환각(Hallucination) 탐지 시스템

문제 정의
- LLM이 그럴듯하지만 틀린 정보를 생성 (hallucination)
- 사용자가 신뢰할 수 있는 정보인지 판별 어려움

해결 방안
LLM 응답 내 주장을 추출 → 신뢰할 수 있는 소스와 대조 → 검증 결과 표시 - Claim Extraction → Evidence Retrieval → Verification - 신뢰도 점수 산출

핵심 기술
```
NLI (Natural Language Inference) 모델
RAG (Wikipedia, 공신력 있는 문서)
LLM API, 신뢰도 점수 알고리즘
Express, MongoDB, 시각화
```

학술적 가치
- AI 신뢰성/안전성 연구의 핵심 주제
- 자동 팩트체크 파이프라인 설계
- 논문 작성 가능: "LLM 응답의 자동 환각 탐지 및 검증 시스템"

예상 개발 기간
- MVP: 6주 / 완성본: 11주

8️⃣ 시스템 장애 로그 분석 AI ⭐ 운영체제 교수 적합

문제 정의
- 서버 장애 시 수천 줄 로그에서 원인 파악 어려움
- 경험 많은 엔지니어의 암묵지에 의존

해결 방안
시스템 로그를 RAG로 인덱싱 + LLM이 근본 원인 분석(RCA) - 이상 패턴 탐지 + 유사 장애 사례 검색 - 단계별 진단 가이드 제공

핵심 기술
```
로그 파서 (syslog, nginx, application logs)
이상 탐지 (통계 기반 또는 Isolation Forest)
RAG (과거 장애 사례 DB), LLM 분석
Express, MongoDB, 실시간 스트리밍
```

학술적 가치
- **AIOps(AI for IT Operations)** 분야 연구
- 시스템 관리 자동화
- 논문 작성 가능: "로그 기반 시스템 장애 자동 진단 시스템"

예상 개발 기간
- MVP: 5주 / 완성본: 10주

9️⃣ 로컬 프라이버시 **AI** 에이전트 **(On-device LLM)**

문제 정의
- 민감한 데이터(일정, 메모, 건강 정보)를 클라우드 AI에 보내기 꺼림
- 완전 로컬 처리 시 성능 제한

해결 방안
경량 LLM(Phi-3, Gemma 2B)을 로컬에서 구동 - 민감도 분류 → 민감 데이터는 로컬, 비민감은 클라우드 - 하이브리드 파이프라인

핵심 기술
```
Ollama / llama.cpp (로컬 LLM 서빙)
데이터 민감도 분류기
하이브리드 라우팅 로직
Express, MongoDB (로컬 데이터 저장)
```

학술적 가치
- 엣지 **AI +** 개인정보 보호 연구
- 리소스 제약 환경에서의 AI 최적화
- 논문 작성 가능: "개인정보 보호를 위한 하이브리드 LLM 아키텍처"

예상 개발 기간
- MVP: 5주 / 완성본: 10주

◆ **API** 문서 자동 생성 및 동기화 시스템

문제 정의
- API 문서가 코드와 동기화되지 않아 신뢰도 하락
- 수동 작성 번거로움

해결 방안
코드베이스 분석 → 엔드포인트/파라미터/응답 추출 → LLM이 자연어 설명 생성 - OpenAPI 스펙 자동 생성 - 코드 변경 시 문서 자동 업데이트

핵심 기술
```
AST 분석 (TypeScript, JavaScript 파서)
Express 라우터 추출, LLM API
OpenAPI/Swagger 생성
Git Hook / GitHub Action 연동
```

학술적 가치
- 코드-문서 동기화 자동화 연구
- 개발자 생산성 향상 도구

- 논문 작성 가능: "LLM 기반 API 문서 자동 생성 및 유지보수 시스템"

예상 개발 기간
- MVP: 4주 / 완성본: 8주

---

3. 프로젝트 추천

3.1 운영체제 교수님 추천 TOP 3

순위	프로젝트	추천 이유
🥇	**LLM 추론 요청 스케줄러 (#1)**	OS 스케줄링 알고리즘(MLFQ, Priority)을 직접 구현하고 LLM 도메인에 적용. 가장 명확한 OS 연관성
🥈	**LLM 응답 시맨틱 캐싱 (#4)**	OS 페이지 교체 알고리즘(LRU, LFU)을 의미론적 캐싱에 확장. 캐시 히트율 실험 가능
🥉	시스템 장애 로그 분석 **AI (#8)**	시스템 로그/커널 이벤트 분석. AIOps라는 실용적 분야 + OS 지식 필수

3.2 학술적 가치 높은 프로젝트 TOP 3

순위	프로젝트	추천 이유
🥇	**LLM 환각 탐지 시스템 (#7)**	AI 신뢰성 연구의 핵심 주제. 논문 작성 가장 용이
🥈	하이브리드 **RAG** 검색 엔진 **(#2)**	검색 기법 비교 실험으로 정량적 성능 평가 가능
🥉	한국어 규정/법령 **RAG (#3)**	도메인 특화 + 한국어 NLP + Knowledge Graph 융합

3.3 실용성 높은 프로젝트 TOP 3

순위	프로젝트	추천 이유
🥇	분산 **LLM** 추론 게이트웨이 **(#6)**	기업에서 바로 사용 가능. 비용 절감 + 안정성 향상
🥈	**API** 문서 자동 생성 **(#10)**	개발자 생산성 도구. 오픈소스로 배포 시 포트폴리오 효과
🥉	한국어 규정/법령 **RAG (#3)**	학교 학칙, 정부 규정 등 실제 사용자에게 직접적 가치 제공

---

4. 상위 프로젝트 상세 명세

4.1 LLM 추론 요청 스케줄러

아키텍처

```
[클라이언트 요청]
        ↓
[게이트웨이 (Express Proxy)]
        ↓
```

```
[요청 분류기] → 우선순위/예산/요청 유형 분석
        ↓
[스케줄러] → MLFQ / Priority Queue / Fair Queue
        ↓
[요청 큐] ← 동적 우선순위 조정
        ↓
[디스패처] → 배칭, Rate Limit 관리
        ↓
[LLM API 호출]
        ↓
[응답 반환 + 메트릭 수집]
```

구현할 스케줄링 알고리즘

알고리즘	설명	적용 시나리오
FCFS	선착순 처리	기본 베이스라인
Priority Queue	우선순위별 처리	VIP 사용자 우선
MLFQ	다단계 피드백 큐	혼합 워크로드
Weighted Fair Queue	가중치 기반 공정 분배	멀티테넌트 환경
Token Budget Scheduler	토큰 예산 기반	비용 최적화

성능 측정 지표

- 평균 대기 시간 (Average Wait Time)
- 처리량 (Throughput)
- 공정성 지수 (Fairness Index)
- 비용 효율 (Cost per Request)

예상 개발 기간

단계	기간	산출물
설계	1주	아키텍처 문서, API 설계
MVP	4주	기본 스케줄러 (FCFS, Priority)
고도화	3주	MLFQ, WFQ 구현
실험	2주	알고리즘별 성능 비교 실험

4.2 하이브리드 RAG 검색 엔진

아키텍처

```
[사용자 질문]
        ↓
    ┌───┴───┐
    ↓       ↓
[Dense]   [Sparse]
(Vector)  (BM25)
    ↓       ↓
    └───┬───┘
        ↓
[RRF Fusion] → 순위 통합
        ↓
[Cross-encoder Re-ranking] → 최종 순위 결정
        ↓
```

```
[Top-K 문서]
        ↓
[LLM + Context] → 답변 생성
```

검색 기법 비교 실험 설계

변수	값
Dense 모델	text-embedding-3-small, text-embedding-3-large
Sparse 모델	BM25, TF-IDF
Fusion 방법	RRF, Linear Combination
Re-ranker	cross-encoder/ms-marco-MiniLM-L-6-v2

평가 지표

- Precision@K, Recall@K
- MRR (Mean Reciprocal Rank)
- nDCG (Normalized Discounted Cumulative Gain)
- 응답 시간

4.3 시스템 장애 로그 분석 AI

아키텍처

```
[로그 소스] → syslog, nginx, application
        ↓
[로그 수집기] → Tail, Filebeat 연동
        ↓
[파서 & 정규화] → 구조화된 JSON
        ↓
[이상 탐지] → 통계 기반 / Isolation Forest
        ↓
[RAG 검색] → 과거 유사 장애 사례 검색
        ↓
[LLM 분석] → 근본 원인 추정 + 해결책 제안
        ↓
[알림 & 대시보드]
```

지원 로그 형식

- Syslog (RFC 5424)
- Nginx access/error log
- Application JSON log
- Docker container log

---

5. 1인 프로젝트 성공 전략

5.1 개발 접근법

1. **MVP** 우선: 핵심 기능 2~3개에 집중하여 동작하는 버전 완성
2. 실험 설계 먼저: 학술적 가치를 위해 비교 실험 설계를 초반에
3. **API** 우선 설계: 프론트엔드는 최소화, 백엔드 로직에 집중
4. 데이터 수집 자동화: 성능 메트릭, 실험 결과 자동 기록

5.2 교수님 설득 포인트

운영체제 교수님
- "OS 이론(스케줄링, 캐싱, 동기화)을 신규 도메인(LLM 서빙)에 적용"
- "알고리즘별 성능 비교 실험으로 정량적 분석 가능"
- "논문 작성 가능한 주제 (학회/저널 투고)"

설득 스크립트 예시

> "LLM 서비스가 급격히 확산되면서, 다수의 동시 요청을 효율적으로 처리하는 것이 중요한 문제가 되었습니다. 본 프로젝트는 운영체제의 프로세스 스케줄링 알고리즘을 LLM 요청 스케줄링에 적용하여, 처리량과 공정성을 개선하는 방안을 연구합니다. 이는 OS 이론의 실용적 확장이며, 성능 비교 실험을 통해 논문으로 발전시킬 수 있습니다."

5.3 주의사항

항목	설명
API 비용	개발 초기에는 GPT-3.5-Turbo, 실험 시에만 GPT-4 사용
실험 데이터셋	공개 데이터셋 활용 (BEIR, KILT, 한국어-LegalBench)
성능 측정	재현 가능한 실험 환경 구축 (Docker, 고정 시드)
코드 품질	테스트 코드 작성, 문서화 철저히

---

6. 기술 스택 가이드

6.1 기본 스택

```
Runtime: Node.js 20+ / Python 3.11+
Framework: Express.js (API), FastAPI (Python 선택 시)
Database: MongoDB Atlas (+ Vector Search)
Queue: BullMQ (Redis 기반)
실시간: Socket.io
```

6.2 AI 스택

```
LLM API: OpenAI API / Claude API / Gemini API
임베딩: OpenAI Embeddings / HuggingFace Sentence Transformers
RAG: LangChain.js / LlamaIndex
로컬 LLM: Ollama, llama.cpp
NLI 모델: HuggingFace Transformers
```

6.3 실험/분석 도구

```
성능 측정: custom metrics, Prometheus
시각화: Chart.js, Plotly, Matplotlib
통계 분석: Python (pandas, scipy)
```

---

7. 참고 자료

학술 자료
- Efficient Memory Management for Large Language Model Serving - vLLM, PagedAttention

- Orca: A Distributed Serving System for Transformer-Based Generative Models
- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

공식 문서

- MongoDB Atlas Vector Search: https://www.mongodb.com/docs/atlas/atlas-vector-search/
- OpenAI API: https://platform.openai.com/docs
- LangChain.js: https://js.langchain.com

트렌드 리포트

- 2025년 홍익대학교 컴퓨터공학과 졸업 프로젝트 기술 백서
- Gartner Top 10 Tech Trends 2025

---

8. 프로젝트 선택 가이드

```
시작 → OS 개념을 직접 적용하고 싶음?
          │
          ├─ YES → 스케줄링 알고리즘 구현 원함?
          │         │
          │         ├─ YES → #1 LLM 추론 스케줄러
          │         │
          │         └─ NO → 캐싱? → #4 시맨틱 캐싱
          │                  로그 분석? → #8 로그 분석 AI
          │
          └─ NO → 학술적 깊이 원함?
                    │
                    ├─ YES → #7 환각 탐지 또는 #2 하이브리드 RAG
                    │
                    └─ NO → 실용성?
                            │
                            ├─ #6 분산 게이트웨이
                            └─ #3 한국어 규정 RAG
```

작성일: 2026년 1월 1일 최종 수정: 2026년 1월 1일 - 학술적 깊이 있는 신한 프로젝트 *10*개로 전면 개편